Section: CS

Understanding Prevalence of Malicious Advertising in Search Engine Results

Chris Humphreys

Problem Statement:

Malicious actors are abusing advertising infrastructure to deliver malware and related malicious content like ClickFix. While the problem is known to exist due to the large number of individuals who have fallen victim to these scams, there has been very little attempt to understand just how prevalent these malicious advertisements are in search engine results in the last roughly ten year time period. Prior to the ~2016 period, there were a number of documented attempts to understand the prevalence of these ads, but I have been able to find very little recent research on this topic. The problem this research will address is the problem of not having a clear picture of how prevalent malvertising currently is, and creating resource efficient tools to help researchers further this study.

Solution Statement:

This research will attempt to find low-resource intensive solutions to measure the prevalence of malicious advertisements in search engine results in order to better understand both how adversaries are hiding malicious advertisements, and to help arm researchers with tools to hunt down and identify malicious ads. This is being done in order to address an area of study that has been neglected for some time and to bring existing, but older techniques up to modern standards. Additionally, this work can be built upon to create even better measures of the problem, or it can be used as a way to measure how effective various attempts to block or fix the malvertising problem are. It will also give us a measurement we can use against the older work to see if the problem of malvertising has increased or decreased since that work was undertaken.

Completed Tasks (Last 2 Weeks) – Public Gihub Repo:

- Wrote the wrapper code so that I can run the full set of queries in a single command. Initially I thought I would need to also write a function to aggregate and grab the totals, but the relatively low number of ads I am pulling in has made this step unnecessary.
- Wrote CLI interface and parameters to allow for execution outside of python interpreter.
- Completed first run of just 100 queries and did successfully pull in and assess a few advertisements. More importantly it really helped to identify bugs and outlier issues with my code that were not as apparent when not running the queries at scale. Fixed these bugs and now have a fully working pipeline from ad collection to assessment.
- Researched into click-fix and spent time deciding if I should just stop pulling in the text of the websites and just use HEAD requests, as this represents a pretty dramatic decrease in proxy bandwith usage, but decided against it. In order to follow the JS redirects, I have to resolve the entire pages, HEAD requests wont bring those in. On the the plus side, decided to add a 4th Oracle for click-fix suspected pages.
- Wrote a 4th Oracle for click-fix pages. Since, as mentioned above, I am bringing in the entire text of the pages anyway, I figured I may as well write up an a search. This ended up being easier than I thought as there are only a few ways to realistically get text into a users clipboard from a web browser. This is exactly what this new Oracle looks like.

- Did a test run of 10,000 queries based on the queries used in the paper "Automated Collection and Analysis of Malware Disseminated via Online Advertising"[16] and was able to get them all to complete! Unfortunately did not get a ton of ads, but I did get enough to confirm that my code is working as expected.
- Thinking perhaps it was my queries that were causing the relatively low number of ad returns, ran a second query list taken from a 2023 paper titled " Data Science Analysis of Malicious Advertisements and Threat Detection Automation for Cybersecurity Progress"[21]. This did result in a larger number of ads, but still not a huge number by any measure.
- Last idea was to ask the AI Google Gemini to create a list of 25 queries that would, based on its training, be the most likely to generate malicious advertisements. Again, better than the first query I ran, but not by a wide margin.
- The real success of all of this testing is I am now confident my application works as expected and is up to the challenge of pulling in a large number of ads.

Last two weeks were good. I was able to add in the fourth Oracle which was a bit of a bonus, and I'm glad its in there. This week and next are really gonna be about doing my final "official" runs of the tool. Since my threading and async programming is paying off by running much faster than I expected, I think I can realistically run all of the query lists above and compare the results of each as well in my final paper.

Questions I have or Issues I'm running into:

So far so good. Things at this point are kind of just coming together!

Methodology Paragraph Summary:

My proposed methodology is to use a (Search Engine Results Page) API provider to collect advertisement URLs from Google. After collecting the advertisement URLs, I will use a residential proxy service in conjunction with a "collector" and "detector" program which I will write to collect the contents of the sites linked to from the ads. Then I will use oracles based on the ones used in earlier research, including lists like the on in the 2014 paper "The Dark Alleys of Madison Avenue: Understanding Malicious Advertisements"[18] to determine if the site is malicious or not. I will create a "Classifier" class to handle this aspect. The results and number of malicious ads identified can be compared with the results of previous studies to better understand the prevalence of modern malvertising.

One of the biggest concerns is avoiding cloaking, or malicious advertisements attempts to hide from researchers and defenders. To bypass this as much as possible I am using the python 'playwright' and 'playwright-stealth' for resolving the ad urls. This uses an actual browser and can evaluate java-script, making it much more resilient to cloaking attempts.

The second concern I have is ensuring that collection and evaluation can be done in a realistic time-frame, and part of this project is seeing if it is possible to write a tool that can do this assessment in an efficient manner. I am writing the collector code (the one that runs the search engine queries) in a multi-threaded way that dramatically speeds up the time this takes. Secondly, I am running my detector code (the following of the ad urls) in a asynchronous manner so that multiple URLS can be resolved concurrently.

Finally, Oracle assessments are done via download URL or domain lists or through measuring telemetry included in the responses (time-codes and strings). This ensures that assessment is completed very quickly and allows for a massive number of assessments.

Timeline:

Week #	Description of Task	Status
W1 (Aug 25 to Aug 31)	Brainstorm Ideas for Project → Firefox Add On	Completed
W1	Research tutorials and lessons on building Firefox AddOns	Complete
W1	Build borderify sample AddOn from Firefox documentation	Complete
W1	Build Beastify AddOn from Firefox documentation	Complete
W2 (Aug 31 to Sept 6)	Brainstorm and determine best course of action based on initial feedback from Dr. Ahmad	Complete
W2	Search ACM articles for anything related to malvertising, read pertinent articles and updated proposal based on findings.	Complete
W2	Search IEEE for any articles related to malvertising, read pertinent articles and update proposal based on findings.	Complete
W2	Do a web search engine based search for non-academic research and findings regarding malvertising.	Complete
W2	Synthesize all findings from literature review and then update proposal and problem statement based on findings.	Complete
W2	Review Dr. Ahmad's feedback again in relation to updated proposal and problem statement ideas. Update to include all feedback.	Complete
W3 (Sept 7 to Sept 14)	Determine which SERP is best suited for my task, set up account and test functionality	Complete
W3	Determine which residential proxy is best suited for my task, set up account and test functionality	Complete
W3	Find a modern working alternative for wepawet oracle. Set up account and test functionality	Complete
W3	Make skeleton of "collector" code that will utilize the SERP API to pull in search engine results.	Complete
W4 (Sept 14 to Sept 20)	Complete code for "Collector" and have it fully functional and ready to be used.	Complete
W4	Find one or more "Domain blacklist" sites I can use to compare ads against to determine if malicious.	Complete
W5 (Sept 21 to Sept 27)	Make skeleton of Detector code.	Complete
W5	Integrate residential proxy with detector code so that requests are less likely to be identified by adversaries	Complete
W6 (Sept 28 to Oct 4)	Complete Detector code and ensure it has the ability to read from the Ad URL's as expected	Complete
W7 (Oct 5 to Oct 11)	Combine detector and collector code into a single application	Complete
W7	Add capability that allows findings to be	Complete

	compared against the “blacklisted Domain Oracle”	
W8 (Oct 12 to Oct 18)	Add capability that allows findings to be compared against the “SSLTimeFrame” malware hash Oracle.	Complete
W8	Add capability that allows findings to be compared against the “URLHaus URL List”	Complete
W9 (Oct 19 to Oct 25)	Combine all Oracle and previous code into one cohesive application that can be used to start scanning.	Complete
W9	Review literature for Evaluation Criteria I can borrow for my own project.	Complete
W9	Do at least 1 successful test run of 1000 pages of search engine results,	Complete
W10 (Oct 26 to Nov 1)	Based on results from W9 test run, update code to solve any identified issues	Complete
W10	Complete and integrate ‘click-fix’ oracle into project.	Complete
W11 (Nov 2 to Nov 8)	Based on test run from W9, determine ideal number of runs to use. (Likely will use all three ad lists I identified above)	Incomplete
W11	Write first draft of Presentation	Incomplete
W11	Run a scan based on number of times decided on above.	Incomplete
W11	Vet results of full run looking for code errors, bugs etc. Fix any identified issues, this is essentially my last chance to identify bugs an d errors.	Incomplete
W12 (Nov 9 to Nov 15)	Do true scan, the results of which will be the final results presented in the paper. – Ensure done by 15th so that I can make presentation video.	Incomplete
W12	Complete final written draft of presentation and any slides needed.	Incomplete
W12	Rehearse presentation 3 times before recording	Incomplete
W12	Submit Final Presentation	Incomplete
W13 (Nov 16 to Nov 22)	Analyze findings, determine conclusions, compare against results from earlier studies, finish first draft of final paper	Incomplete
W13	Complete second draft of Final Paper.	Incomplete
W14 (Nov 23 to Nov 29)	Thanksgiving	
W15 (Nov 30 to Dec 6)	Write final draft of paper	Incomplete
W15	Submit!	Incomplete

Evaluation:

I don’t want to base my evaluation on how many malicious ads I find, as that number could be low for a number of reasons beyond my control. Instead, I think I should measure my success by the number of ad pages (query executions) I am able to evaluate, and then how accurate that assessment turns out to be.

Success for this project would look like being able to evaluate 10k search engine result pages against my Oracles and successfully labeling any ads which are malicious as being such. That of course begs the question of how will I know how accurate I was. I think my plan is to grab 100 random analyzed ad pages, that have been through my oracles, and manually check the findings by hand. I can compare this to the number my Oracles identified as malicious and get an accuracy percentage, from that I will extrapolate it and use that as my total accuracy. Knowing this plan, I think the evaluation criteria below is reasonable.

- # of Queries Evaluated
  - < 500 Poor
  - 500 – 1000 Adequate
  - 1000 – 5000 Good
  - 5000 – 10000 Great
  - >= 10000 Excellent
- Accuracy %
  - < 50% Poor
  - 50% – 70% Adequate
  - 70% - 90% Good
  - 90% - 99% Great
  - 100% Excellent

I am adding a third criteria as well, which is number of Oracles implemented. This is a good evaluation measure as the more Oracles integrated, the more likely malicious ads will be identified, and the better the overall coverage of the tool.

- # of Oracles
  - 1 Poor
  - 2 Good
  - 3 Great
  - 4 Excellent

Report Outline:

Draft Report (Right Now my draft is basically my outline: This will flesh out as time passes):

I. Abstract:

II. Introduction:

A. Define Problem (Lack of studies on search engine malicious ad prevalence since 2016)

B. Clearly State Negative repercussions of not studying this; orgs and individuals are targeted and victimized.

III. Past Research

A. Review previous studies that directly mirror what I will be doing or that I borrowed and built up the methodology

1. The Dark Alleys of Madison Avenue: Understanding Malicious Advertisements.

a. Borrowed Idea of Oracles and took inspiration for some of mine

2. Automated Collection and Analysis of Malware Disseminated via Online Advertising

a. this is where I will take my search queires from. They also have some system design items I took inspiration I will elaborate on

B. Review Related malvertising studies that are less directly related

1. All the ones that use machine learning or novel methods of identification

2. Articles that look for malicious ads, but in different contexts like browser extensions

C. Discuss the relative lack of work over the past 5 to 10 years

IV. Methodology

A. SERP and Pulling google Ads

1. Issues with rate limiting

2. SERP providers and other options

3. Discussion of my decision to use SERP and why

B. Residential proxy

1. Cloaking and discussion of why a proxy is even necessary

2. Other options and why I went with a commercial proxy

3. Issues not resolved by proxy usage

C. Query selection process

1. discuss previous work and what I copied

2. discuss changes I made and why

a. Clickfix searching

b. New technologies and terms since last paper

D. System Design

1. Review of how my system works

a. Python object “Collector”, collects the google ad links

b. Python object “Detector”, detects the actual ad website, ideally bypassing cloaking protections

c. Python object “Classifier”, runs the oracles which determine if the ads are malicious or not.

i. Domain BlockList Oracle

ii. URLHaus Oracle

iii. SSL Oracle

iv. ClickFix Oracle

V. Results

A. Findings

1. After a run of ideally about 10k queries (may need to adjust based on speed) how many ads were identified

2. Of identified ads how many were deemed malicious and by which oracle

a. Domain Oracle Results

b. SSL Oracle Results

c. URLHaus Oracle Results

d. Clickfix Oracle Results

3. Optional: Findings at different times and locations

B. Comparisons to previous work

1. What findings did other papers have, and even if the match is not 1:1, can we infer findings from what we do see.

2. Highlight significant deviations from previous work and attempt to explain.

C. Future Work

1. Better ways to avoid cloaking

2. How to build on this work

VI. Conclusion

A. Did I find ads, and how many

B. Was this worthwhile, and more importantly, is it worth continuing, or did I learn that this method of detection is not longer valuable.

References:

[1] anon. Malvertising What is malvertising and how to protect yourself. Malwarebytes. Retrieved September 65 2026 from https://www.malwarebytes.com/malvertising

[2] anon. 2026. Malvertising & Ad Quality Index - Risky Business. Retrieved September 6, 2026 from https://www.confiant.com/hubfs/Team%20-%20Marketing/THE%20MAQ%20INDEX%202026_%20RISKY%20BUSINESS.pdf

[3] Contributors to Wikimedia projects. 2010. Malvertising. Wikipedia. Retrieved September 6, 2026 from https://en.wikipedia.org/wiki/Malvertising

[4] Gadget Hacks. 2026. Ad Blockers That Still Work in Chrome: 4 Ways to Block Ads. Gadget Hacks. Retrieved September 6, 2026 from https://android.gadgethacks.com/how-to/ad-blockers-that-still-work-in-chrome-4-ways-to-block-ads/

[5] David Kasabji. 2025. From SEO Poisoning to Malware Deployment: Malvertising campaign uncovered. Conscia. Retrieved September 5, 2026 from https://conscia.com/blog/from-seo-poisoning-to-malware-deployment-malvertising-campaign-uncovered/

[6] Jérôme Segura. 2016. A Look Behind The Skype Malvertising Campaign | Malwarebytes Labs. Malwarebytes. Retrieved September 5, 2026 from https://www.malwarebytes.com/blog/news/2016/02/a-look-behind-the-skype-malvertising-campaign

[7] Dhanur Sehgal. 2026. What Is Malvertising? Examples, Risks & Prevention. The Blockify Blog. Retrieved September 5, 2026 from https://getblockify.com/blog/malvertising/

[8] Michael Steele. 2026. SourTrade: Browser-Assembled Malware Delivered Through Malvertising. Confiant. Retrieved September 5, 2026 from https://blog.confiant.com/p/sourtrade-browser-assembled-malware

[9] George Thomas. 2015. Malvertising up 325% - Are the AdBlockers Working?. Retrieved September 6, 2026 from https://web.archive.org/web/20160130161632/http://blog.check-and-secure.com/091015-malvertising-up-325-are-adblockers-working/

[10] Shubham Kumar, Siddharth Swarup Rautaray, and Manjusha Pandey. 2017. Malvertising: A case study based on analysis of possible solutions. In 2017 International Conference on Inventive Computing and Informatics (ICICI), November 2017. IEEE, 288–291. https://doi.org/10.1109/icici.2017.8365356

[11] Joshua Rolon, Cheryl Hinds, and Felicia Doswell. 2019. SpartanShield: A Layered Defense against Malvertising. In 2019 International Conference on Computational Science and Computational Intelligence (CSCI), December 2019. IEEE, 185–190. https://doi.org/10.1109/csci49370.2019.00038

[12] Eric Blancaflor, Jon Eril R. Aparize, Joyous M. Geronimo, Ethan Joseph S. Gonzales, and Joyful L. Morilla. 2024. Malvertising in AI-Powered Chat Platforms: Utilizing Integrated Artificial Intelligence Solutions for Threat Detection and Mitigation. In 2024 8th International Conference on Communications and Future Internet (ICCFI), July 19, 2024. IEEE, 67–74. https://doi.org/10.1109/iccfi63302.2024.00018

[13] Ankita Kumari and Ishu Sharma. 2024. Mitigating Malvertising Threats: An Exploration of Machine Learning Classification Algorithms for Effective Detection. In 2024 International Conference on Advancements in Smart, Secure and Intelligent Computing (ASSIC), January 27, 2024. IEEE, 1–6. https://doi.org/10.1109/assic60049.2024.10508033

[14] Chin-Tser Huang, Muhammad N. Sakib, Charles Kamhoua, Kevin Kwiat, and Laurent Njilla. 2017. A game theoretic approach for inspecting web-based malvertising. In 2017 IEEE International Conference on Communications (ICC), May 2017. IEEE, 1–6. https://doi.org/10.1109/icc.2017.7996807

[15] Chin-Tser Huang, Muhammad N. Sakib, Charles A. Kamhoua, Kevin A. Kwiat, and Laurent Njilla. 2020. A Bayesian Game Theoretic Approach for Inspecting Web-Based Malvertising. IEEE Trans. Dependable and Secure Comput. 17, 6 (November 2020), 1257–1268. https://doi.org/10.1109/tdsc.2018.2866821

[16] Muhammad N. Sakib and Chin-Tser Huang. 2015. Automated Collection and Analysis of Malware Disseminated via Online Advertising. In 2015 IEEE Trustcom/BigDataSE/ISPA, August 2015. IEEE, 1411–1416. https://doi.org/10.1109/trustcom.2015.539

[17] Tiliang Zhang, Hua Zhang, and Fei Gao. 2013. A Malicious Advertising Detection Scheme Based on the Depth of URL Strategy. In 2013 Sixth International Symposium on Computational Intelligence and Design, October 2013. IEEE, 57–60. https://doi.org/10.1109/iscid.2013.128

[18] Apostolis Zarras, Alexandros Kapravelos, Gianluca Stringhini, Thorsten Holz, Christopher Kruegel, and Giovanni Vigna. 2014. The Dark Alleys of Madison Avenue: Understanding Malicious Advertisements. In Proceedings of the 2014 Conference on Internet Measurement Conference, November 05, 2014. ACM, 373–380. https://doi.org/10.1145/2663716.2663719

[19] Xinyu Xing, Wei Meng, Byoungyoung Lee, Udi Weinsberg, Anmol Sheth, Roberto Perdisci, and Wenke Lee. 2015. Understanding Malvertising Through Ad-Injecting Browser Extensions. In Proceedings of the 24th International Conference on World Wide Web, May 18, 2015. International World Wide Web Conferences Steering Committee, 1286–1295. https://doi.org/10.1145/2736277.2741630

[20] Zhou Li, Kehuan Zhang, Yinglian Xie, Fang Yu, and XiaoFeng Wang. 2012. Knowing your enemy. In Proceedings of the 2012 ACM conference on Computer and communications security, October 16, 2012. ACM, 674–686. https://doi.org/10.1145/2382196.2382267

[21] S. Nguyen and D. Bein, "Data Science Analysis of Malicious Advertisements and Threat Detection Automation for Cybersecurity Progress," *2023 IEEE 13th Annual Computing and Communication Workshop and Conference (CCWC)*, Las Vegas, NV, USA, 2023, pp. 0695-0704, doi: 10.1109/CCWC57344.2023.10099325.

Appendix

Query Lists:

Automated Collection and Analysis of Malware Disseminated via Online Advertising [16]

Qualifiers – [Free, Download, Watch, Stream, Hack]

Content – [Movies, Ebook, Pdf, Music, Mp3, Wallpaper, Fonts, News, Sports, Highlights, Software, Freeware, Cracks, Password]

Data Science Analysis of Malicious Advertisements and Threat Detection Automation for Cybersecurity Progress[21]

“nft”, “cloud computing services,” “malware, ” “app development”, “cryptocurrency,” “web design”, “web development,” “resume building,” “infographic maker,” “job search sites, ” “job search,” “internship search”, “online news,” “academic poster template,” “video editing software,” “photo editing

software," "color correction software," "photoshop software," "augmented reality software," "artificial intelligence software," and "adobe software."

AI Generated Query List (Google Gemini)

"download putty", "winscp free download", "advanced ip scanner download", "filezilla client download", "notepad++ download", "windirstat download", "obs studio download", "rufus bootable usb download", "anydesk download", "teamviewer free download", "cisco anyconnect vpn download", "zoom workplace desktop app", "slack desktop app download", "microsoft teams download", "phantom wallet chrome extension", "metamask extension download", "ledger live desktop download", "exodus wallet download", "winrar free download", "7-zip download", "vlc media player download", "freecad download", "blender 3d download", "wireshark download", "postman api client download"